

# AgentRaaS

The reliability layer for AI agents that take real actions: every call runs once, within the rules you set, risky calls wait for a human, and every action is on the record.

THE PROBLEM

# A retry can become a second charge

1

### The API call succeeds

Stripe charges the card. The response is on its way back.

2

### The response is lost

A timeout, a dropped connection, a crashed worker.

3

### The agent retries

The card is charged twice. The customer finds out first.

The most common failure, not the only one: agents also act outside their remit (a prompt-injected email to a stranger), loop on the same call, and leave no record anyone can audit.

WHY NOW

# Agents have started acting, not just answering

### Tools gave agents hands

Through tool calling and MCP, agents now move money, send email, update CRMs and file tickets on their own.

### Retries are built in

Frameworks, queues and HTTP clients retry automatically, so duplicate actions are the default failure, not an edge case.

### Every team rebuilds it

Idempotency, approval steps and action logs get hand-built per agent, per API. There is no shared layer for it yet.

WHAT WE OFFER

# Six controls in one layer

### Runs once

Atomic dedup, near-duplicate matching, and step checkpoints so a failed run resumes, not repeats.

### Human approval

Risky calls wait for a person to approve in Slack, Teams, Telegram, Discord, email or the dashboard.

### Policies and limits

What each agent may do, daily money caps, allowed hours, call caps and loop limits.

### Resilience

Circuit breaking, safe retries, a replayable dead-letter queue, and one-click undo of a charge or message.

### Audit and identity

Tamper-evident logs, OpenTelemetry export, scoped agent identities, and record and replay for testing.

### Data safety

PII redaction, prompt-injection filtering on tool output, and schema drift alerts.

HOW IT WORKS

# The life of one agent call

01

## Identify

An API key or a scoped agent token says which agent is acting, for which customer.

02

## Validate

Per-customer rules check the payload, such as amount limits or a refund above the balance.

03

## Claim

An atomic dedup slot is taken. If it already exists, the stored result goes back instead.

04

## Govern

Spend caps and loop checks run; a call that needs approval waits for a human.

05

## Execute

Forwarded once with the customer's encrypted credentials, with retries and a circuit breaker.

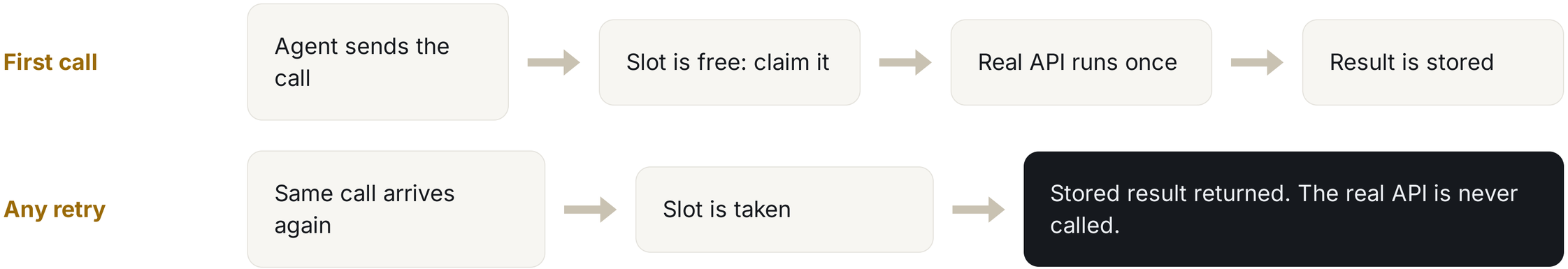
06

## Record

Logged with the agent's identity. A failure lands in a queue for replay, never silently lost.

THE CORE MECHANISM

# How exactly-once works




The slot is claimed atomically in shared storage, so it holds across retries, crashed workers and many machines at once, and it is proven under real concurrent load.

WHEN SOMETHING GOES WRONG

# Undo it, or test it before it ships

Undo log

## Reverse an action, once

A Stripe charge is refunded, a Slack message deleted, a custom action reversed by the action you name.

Kept for 30 days. Two clicks at the same moment still reverse it only once.

In the dashboard: Monitor, Undo actions.

Record and replay

## Test against real answers

Record a run once against the real providers, then replay it in CI or after a prompt change.

Every call is answered from the recording; nothing reaches the provider. A changed payload or a call the recording lacks fails the test.

Webhook, Python and TypeScript SDK, and MCP.

GETTING STARTED

# Start free, without an account

Find the bug

## Chaos tester

Shows the calls an agent would run twice, and agentraas scan checks an MCP server. Free, no signup, both run in CI.

Fix it in code

## Library and MCP wrap

An exactly-once decorator, and one command that protects any MCP server.

Run it as a platform

## Self-host it, free

Exactly-once, policies, approvals, SSO, undo and the audit trail on Linux, macOS or Windows, no action limit.

**Connect by** webhook, Python or TypeScript SDK, or MCP

**Works with** n8n (community node), LangChain, CrewAI and any HTTP client

**Found through** PyPI, npm, GitHub and the official MCP Registry

BUILT FOR THE CRITICAL PATH

# Fast, open and self-hostable

## 2.1 to 2.4 ms

added per call at the median. A retried duplicate is answered from cache and never reaches the API.

### Rust core

An Axum and Tokio gateway on Postgres and Redis, serving production today.

### Open source core

AGPL-3.0 server and every line of code public: you can read the code that guards your payments.

### Runs where the data is

One command installs it on the customer's own servers: Linux, macOS or Windows.

Latency: one machine (Intel Core i3-12100), local stack, 500 sequential calls, three runs; script in the repo.

OPEN SOURCE

# Open source, free to self-host

| Part | License | In plain words |
| --- | --- | --- |
| **The server** | AGPL-3.0 | Use, change and run it for anything. Share your changes if you offer them over a network. |
| **Team and Enterprise features** | Source-available | Team features are free in production on your own servers. Enterprise features need a license. |
| **SDKs and tools** | MIT or Apache-2.0 | No conditions beyond keeping the notice. |

**Self-hosting needs no license key and has no action limit, approvals and SSO included.**

Details: LICENSE.md in github.com/sumedhchatse/agentraas

AVAILABLE TODAY

# Built, live and ready to try

## What you get

- A Rust gateway running in production
- 31 services ready to connect: paste a key, copy a URL
- Python SDK on PyPI, TypeScript SDK on npm
- Webhook, SDK and MCP integration
- Open source (AGPL), free to self-host on Linux, macOS or Windows
- Helm chart, rules as code in Git

## Where to find it

- Live demo, no signup: agentraas.io
- Docs: agentraas.io/docs
- Install: pip install agentraas
- Or: npm install agentraas
- Source: github.com/sumedhchatse/agentraas
- Official MCP Registry: io.github.sumedhchatse/agentraas

# Try it on your agent, free

See it live, no signup: agentraas.io

Find duplicate calls: agentraas chaos

Install: pip or npm install agentraas

Self-host free: agentraas.io/self-host

agentraas.io

GitHub